

TaskFlow

SUMMARY

TaskFlow is a full-stack task management application built as a technical assessment for ElectroPi using ASP.NET Core 10, Angular 21, and SQL Server. It allows you to manage projects and tasks through a clean, responsive UI with full CRUD support, task status updates, and task filtering.

PROJECTS

TaskFlow — Task Manager | Full-stack task management application built as a technical assessment for ElectroPi.

- Project Management — create, list, view details, edit, and delete projects
- Task Management — create, list, view, edit, and delete tasks
- Task Status Updates — change task status directly without opening a separate page
- Status Filtering — filter tasks by status across all projects
- Project Dashboard — view a project with its tasks and quick stats
- Validation — backend validation with FluentValidation and frontend form validation
- Error Handling — centralized exception handling middleware with structured JSON responses
- API Documentation — interactive API reference via Scalar
- Seed Data — sample projects and tasks are inserted on first run
- Docker Support — backend can be run in a container
- Backend follows a simplified Clean Architecture with a modular monolith structure: TaskManager.API, TaskManager.Core, TaskManager.Infrastructure
- Projects and Tasks are organized as independent feature modules
- Frontend uses Angular standalone components with routed feature pages and shared services/models for API communication
- Global exception middleware logs exceptions, returns consistent JSON responses, and prevents leaking internal implementation details
- Interactive API documentation available at <http://localhost:5045/scalar/v1> in Development mode

TECHNICAL SKILLS

ASP.NET Core 10

C#

Entity Framework Core 10 (Code First)

SQL Server

FluentValidation

AutoMapper

Scalar.AspNetCore

Angular 21

TypeScript

CSS Custom Properties

Docker

API OVERVIEW

Base URL: <http://localhost:5045/api>

Projects endpoints: GET /Projects — List all projects GET /Projects/{id} — Get a single project GET /Projects/{id}/tasks — Get tasks for a specific project POST /Projects — Create a project PUT /Projects/{id} — Update a project DELETE /Projects/{id} — Delete a project

Tasks endpoints: GET /Tasks — List all tasks GET /Tasks/{id} — Get a single task GET /Tasks/status/{status} — Filter tasks by status POST /Tasks — Create a task PUT /Tasks/{id} — Update a task PATCH /Tasks/{id}/status — Update task status only DELETE /Tasks/{id} — Delete a task

SETUP INSTRUCTIONS

Prerequisites: .NET 10 SDK, Node.js 18+, SQL Server or LocalDB, Angular CLI.

Backend: 1. Update connection string in Backend/TaskManager.API/TaskManager.API/appsettings.json. 2. Run API: cd Backend/TaskManager.API/TaskManager.API dotnet run Database is created and migrated automatically on startup. 3. Optional manual migrations: dotnet ef database update --project ../TaskManager.Infrastructure

Frontend: 1. cd FrontEnd/task-Manager-ui npm install 2. npm start Frontend runs at http://localhost:4200 and sends requests to http://localhost:5045/api.

Docker: cd Backend/TaskManager.API docker build -f TaskManager.API/Dockerfile -t taskmanager-api . docker run -p 5045:8080 taskmanager-api

CONFIGURATION

CORS is configured in Program.cs. Database connection is stored in appsettings.json. Frontend API base URL is configured in environment.ts. Auto-migration runs on startup.

DESIGN DECISIONS

Simplified Clean Architecture keeps the solution organized without over-engineering. Feature-based structure separates Projects and Tasks clearly. Repository + Service Pattern keeps controllers thin and business logic centralized. FluentValidation keeps DTOs clean and validation rules expressive. Result Pattern provides consistent success/failure returns. Angular Standalone Components reduce boilerplate. Global Exception Middleware handles unhandled errors in one place. Seed Data helps demonstrate the application quickly after first run.

FUTURE IMPROVEMENTS

Add authentication and authorization with JWT; add pagination and search; add comprehensive unit and integration tests; add CI/CD with GitHub Actions; add real-time updates with SignalR; add drag-and-drop task ordering; extend modular structure into separate microservices if the app grows.

LICENSE

This project was built as a technical assessment for ElectroPi.